

Configure User Permissions and Policies

Task-1.Create a comparison table showing which security responsibilities belong to the cloud provider and which belong to the user for a food delivery app like Zomato running on AWS, Azure, or Google Cloud.

Security area	Cloud provider (AWS)	User / Zomato
Physical data centers	Secures buildings, physical servers, networking equipment, and data-center access.	No responsibility for physical infrastructure.
Hardware & virtualization	Maintains and secures underlying hardware and virtualization layer.	No responsibility for the underlying hardware.
Operating system	For managed services, AWS handles much of the underlying infrastructure; for EC2, the customer must patch the guest OS.	For EC2, Zomato must install security patches and secure the OS.
Applications	Provides security features and infrastructure for AWS services.	Must secure application code, APIs, authentication, and vulnerabilities.
Data	Provides encryption and security tools.	Must classify, encrypt, back up, and properly protect customer/order/payment data.
IAM & permissions	Provides AWS IAM and access-control capabilities.	Must create users/roles and apply least-privilege permissions.
Network security	Provides VPC, security groups, firewalls, and related services.	Must configure them correctly and restrict unnecessary access.
Monitoring	Provides services such as CloudTrail, GuardDuty, and Security Hub.	Must enable/configure monitoring and respond to security alerts.

Task-2.Sign up for a free trial on AWS, Azure, or Google Cloud, and create a new user account with limited permissions (for example, only allow access to storage buckets or databases, but not to billing or admin settings).

- ❓ Sign in to the **AWS Management Console**.
- ❓ Search for and open **IAM (Identity and Access Management)**.
- ❓ Go to **Users** → **Create user**.
- ❓ Enter a name such as User1.
- ❓ Choose the appropriate sign-in/credential option required for your lab.
- ❓ Under permissions, choose **Add permissions**.
- ❓ Select **Attach policies directly**.
- ❓ Search for an appropriate limited policy, such as **AmazonS3ReadOnlyAccess**,
- ❓ Review the permissions and click **Create user**.

The screenshot shows the AWS IAM console for 'User1'. The left sidebar contains navigation links for Identity and Access Management (IAM), Access Management, and Access reports. The main content area is titled 'User1' and includes a 'Summary' section with details like ARN, creation date, and console access status. Below this, the 'Permissions' tab is active, showing a table of 'Permissions policies (1)'. The table lists the 'AmazonS3ReadOnlyAccess' policy, which is AWS managed and attached directly. There are also buttons for 'Simulate', 'Remove', and 'Add permissions'.

The screenshot shows the AWS S3 console with an 'Upload failed' error message. The error message is displayed in a red banner at the top. Below the banner, there is a table titled 'Files and folders (1 total, 519.3 KB)'. The table has columns for Name, Folder, Type, Size, Status, and Error. The table contains one row with the file name 'Screenshot 2026-05-27 1...', a status of 'Failed', and an error of 'Access denied'.

Task-3. Configure a custom policy in your chosen cloud platform that allows a user to upload images to a storage bucket but denies them permission to delete any files. Look for storage service permissions such as 'PutObject' and 'DeleteObject' in the policy editor.

Create an IAM policy that allows the user to upload objects but does not delete any file

Steps

1. Open **AWS Console** → **IAM**.
2. Select **Policies** → **Create policy**.
3. Choose **JSON**.
4. Enter a policy
5. Click **Next** and give the policy a name such

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "Statement1",  
      "Effect": "Allow",  
      "Action": [  
        "s3:PutObject"  
      ],  
      "Resource": "arn:aws:s3:::ahfpl/*"  
    },  
    {  
      "Sid": "Statement2",  
      "Effect": "Deny",  
      "Action": [  

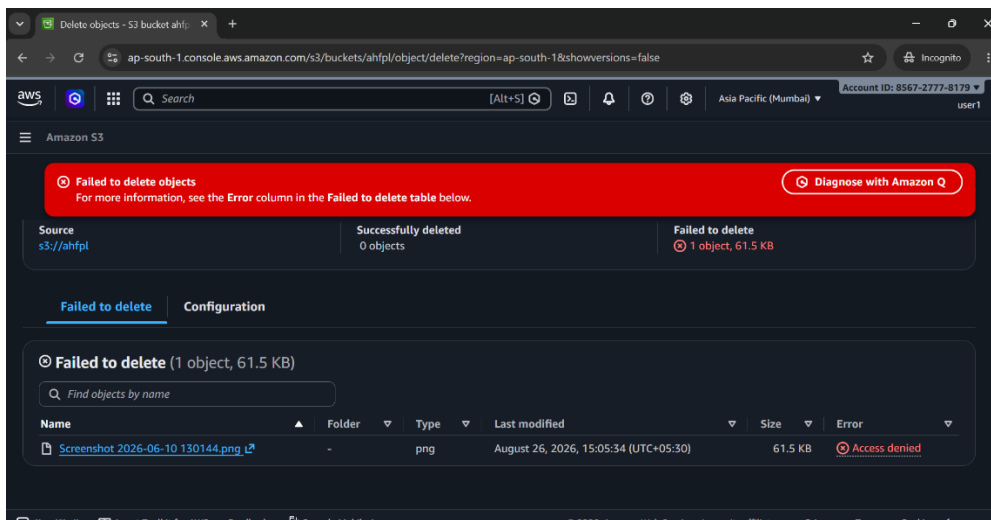
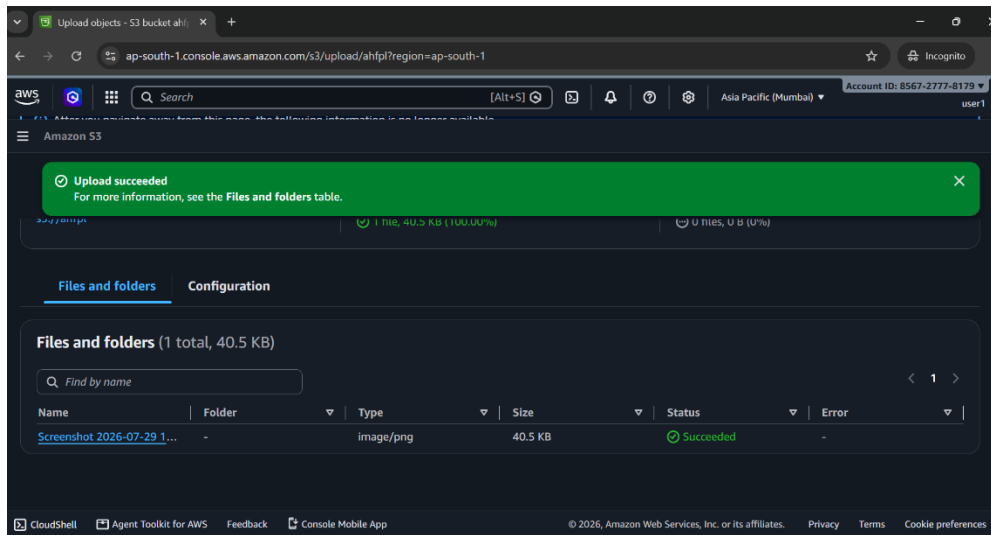
```

```

        "s3:DeleteObject"
    ],
    "Resource": "arn:aws:s3:::ahfpl/*"
}
]
}

```

6. Create the policy.
7. Attach it to the limited-permission IAM user or role.

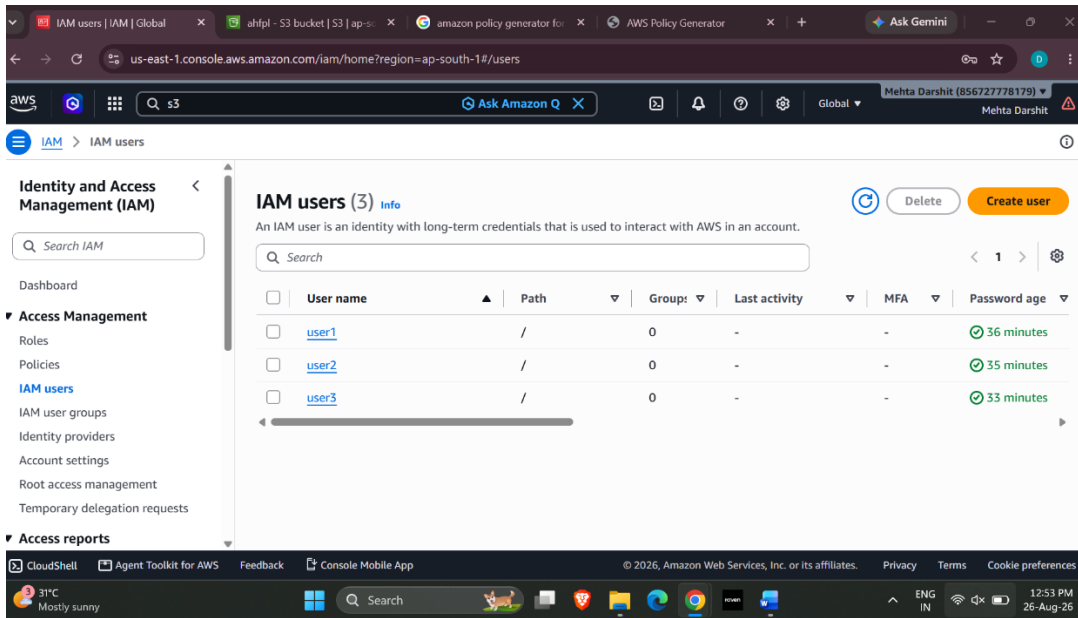


Task-4. Simulate a real-world scenario: Imagine you are part of a team managing a cricket ticket booking app (like BookMyShow). Assign the following roles using your cloud provider's IAM or equivalent: one user can only view booking data, another can edit booking data, and a third can manage user accounts. Use built-in roles or create custom ones if needed.

User	Role	Required permissions
User 1	Booking Viewer	Can view booking/customer information but cannot modify it.
User 2	Booking Editor	Can view and create/update booking information but cannot manage IAM users.
User 3	User Account Manager	Can manage application user accounts but should not automatically have access to sensitive AWS administration.

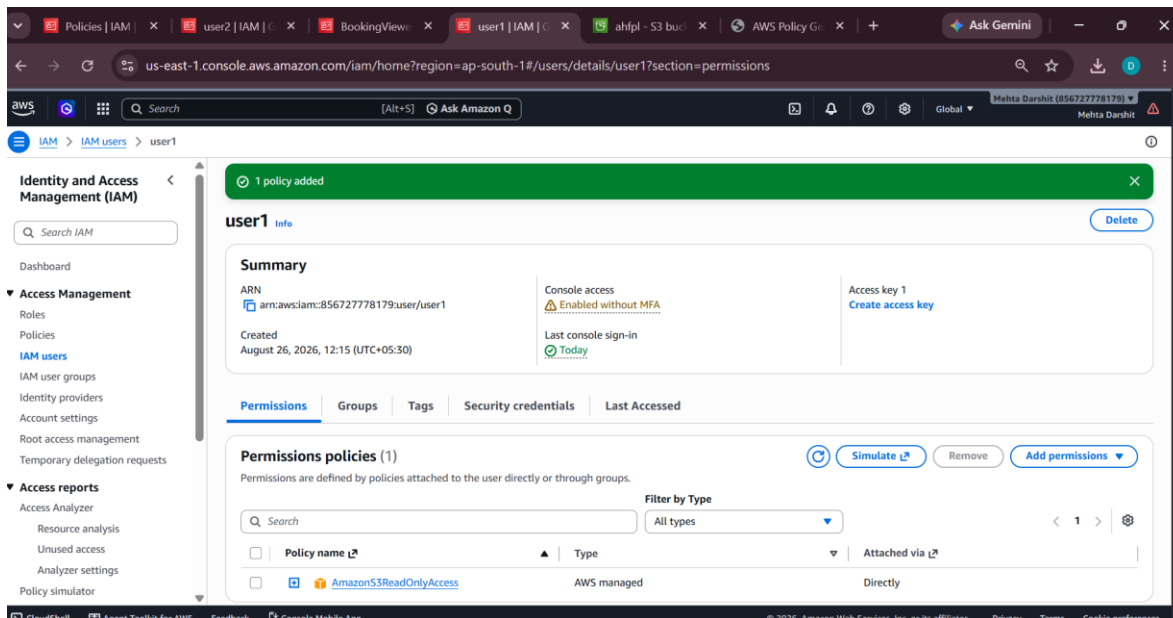
How to configure this in AWS

1. Open **IAM** → **Policies**.
2. Create a BookingViewer policy with only the required **read** permissions.
3. Create a BookingEditor policy with the required **read/write** permissions.
4. Create a UserAccountManager policy containing only the IAM/application-account-management permissions that the third user actually needs.
5. Create IAM roles or users and attach the corresponding policies.
6. Test each identity to make sure it can perform its assigned tasks but receives **Access Denied** for unauthorized actions.



Three IAM users were created with different levels of access. The Booking Viewer can only view booking data, the Booking Editor can view and modify booking data, and the User Account Manager can manage user accounts. This demonstrates role-based access control and the principle of least privilege.

USER1 Policy :-

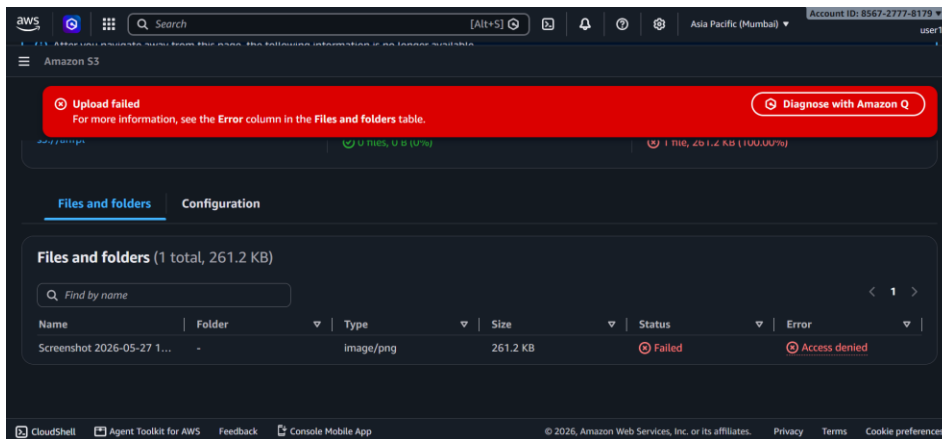


User 1 can:

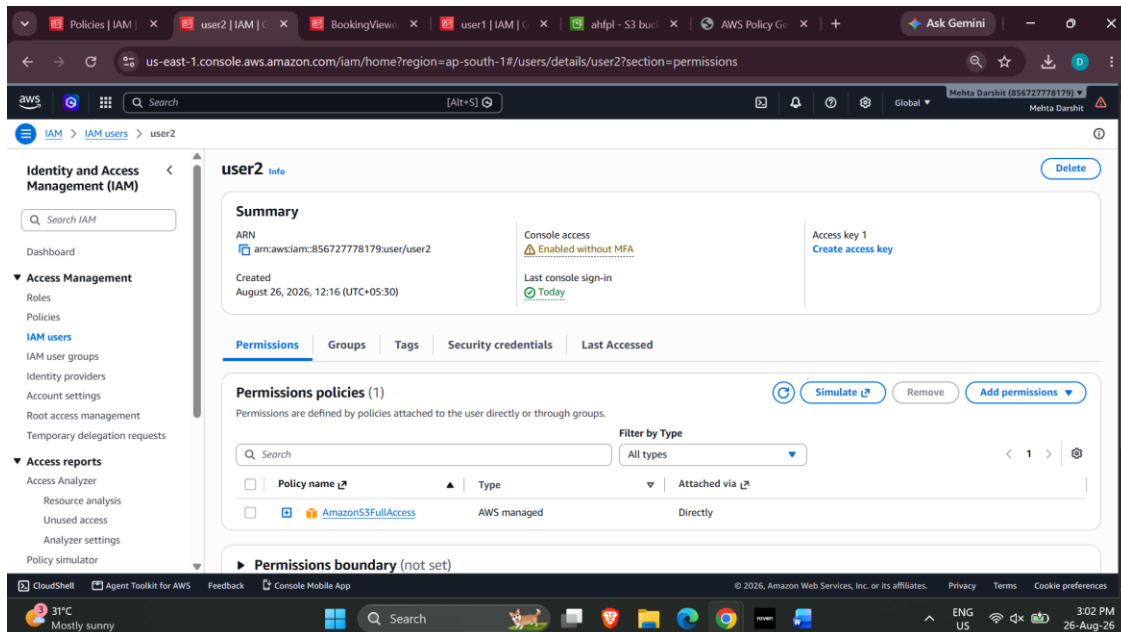
- View/list booking files
- Download booking data

User 1 cannot:

- Upload or edit booking data
- Delete booking data
- Manage users
- Access billing



USER2 Policy:

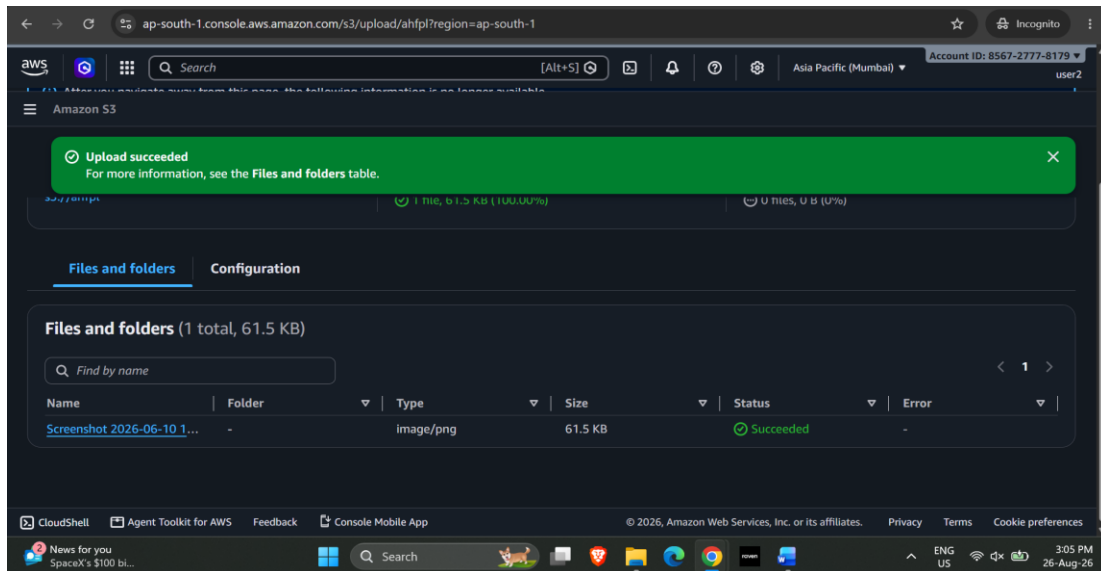


User 2 can:

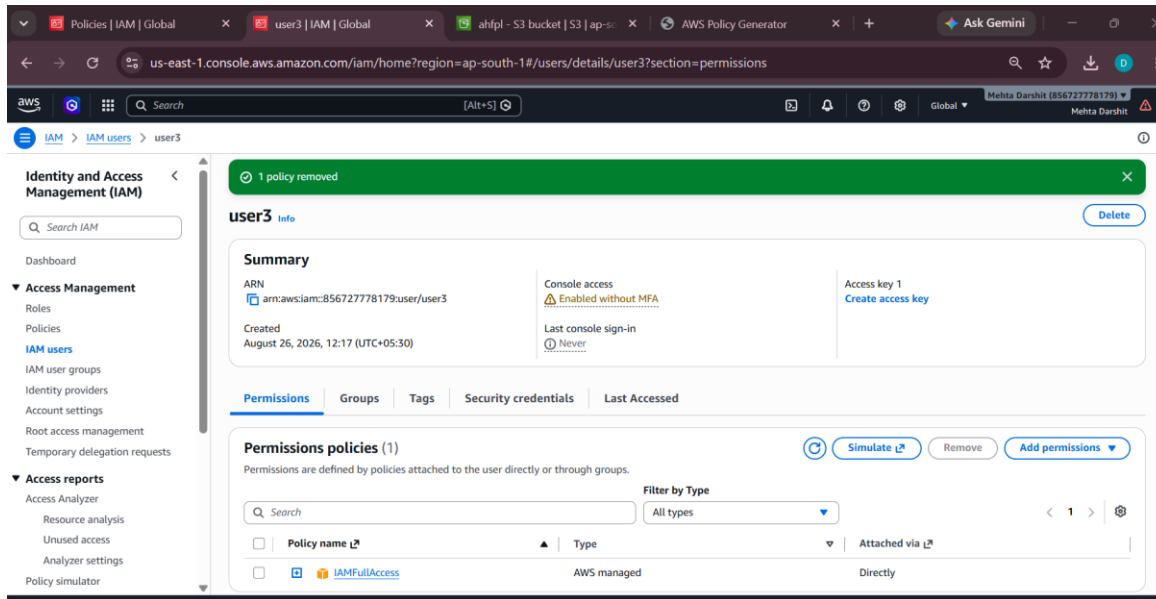
- View booking data
- Upload new booking files
- Update/overwrite booking files

User 2 cannot:

- Manage IAM users
- Change AWS policies
- Access billing
- Manage EC2 infrastructure



USER3 Policy:-



What this user can do

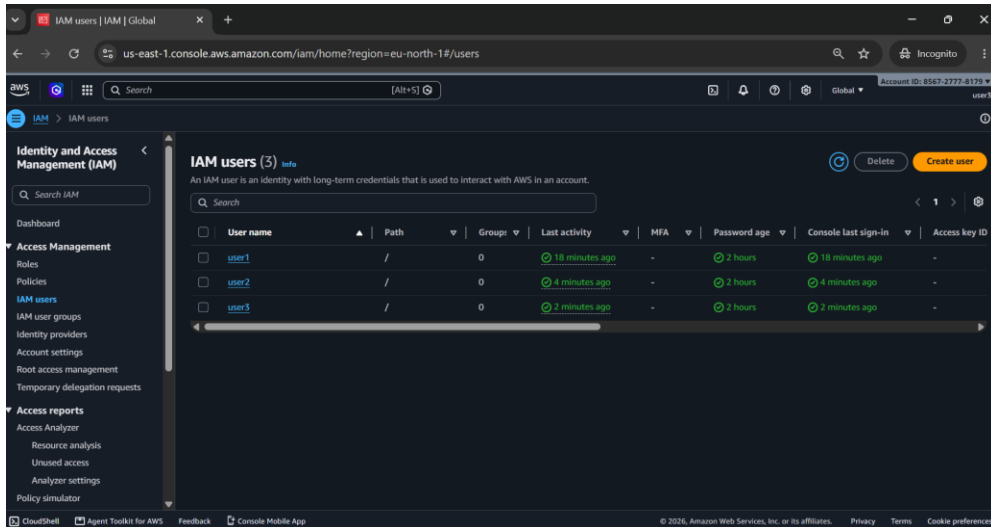
The user can:

- Create IAM users
- View/list users
- Update user information
- Delete users
- Create/delete login profiles
- Update login profiles

What this user cannot do

Because we haven't granted these permissions, they **cannot**:

- Manage billing
- Create/delete EC2 instances
- Access S3 buckets
- Modify IAM policies
- Give themselves Administrator access
- Manage groups or roles



Task-5. Find and describe one real security incident from the last 3 years where a cloud misconfiguration (wrong permissions or policies) exposed user data. Explain in 3-4 lines how applying the Shared Responsibility Model could have prevented it.

Example: Exposed AWS environments — 2024

In August 2024, Unit 42 researchers reported a large campaign involving approximately **110,000 domains** where exposed .env files contained AWS credentials, database credentials, API keys, and other secrets. Attackers used the exposed credentials to gain access to cloud environments and potentially exfiltrate data.

How the Shared Responsibility Model could have helped: AWS provides the underlying security and IAM controls, but customers are responsible for securely configuring their applications, credentials, and permissions. Using least-privilege IAM policies, avoiding exposed credentials, rotating/using short-lived credentials, and monitoring unusual activity could have significantly reduced the risk. AWS itself emphasizes that unintended customer configurations fall within the customer's side of the shared-responsibility model.